

# **Project Proposal:**

## **Predicting HSBC Weekly Closing Price via Machine Learning Regression**

**Course:** ELEC3544 Data Science with Foundation Models

**Team Members:** Group E

Yu Zhenghao 3036263270

Wu Hao 3036226404

Bai Yu3036262123

### **1. Abstract**

This project focuses on the 1-week-ahead prediction of HSBC Holdings' weekly closing stock price by constructing a hierarchical machine learning regression pipeline, with publicly available multi-dimensional market time-series data from 2019 to 2025 as the research basis. Taking the current week's market features as input variables, the study designs and validates regression models ranging from basic OLS to advanced gradient boosting models, and benchmarks all model performance against the NPB. Beyond achieving accurate price prediction, the project identifies the key driving features of HSBC's stock price from four dimensions (firm-specific data, banking peer performance, macroeconomic indicators and market sentiment), quantifies the marginal value of machine learning in financial time-series forecasting, and strictly adheres to time-series analysis best practices to avoid lookahead bias and data leakage. The project adopts rigorous data handling protocols, model evaluation metrics and risk mitigation strategies, and ensures full reproducibility of the research process and results through standardized code documentation and version control. The research outcomes are expected to provide empirical references for understanding the fluctuation rules of cross-listed global bank stocks and establish a standardized end-to-end pipeline for financial time-series prediction.

### **2. Project Goals**

The primary goal of this project is to construct and validate a series of regression

models for the 1-week-ahead prediction of HSBC Holdings' weekly closing stock price, utilizing publicly accessible market time-series data covering the period 2019 to 2025. The core framework of the models is designed to map the current week's multi-dimensional market feature data to the predicted closing price of HSBC in the subsequent week, thereby implementing a targeted time-series forecasting solution for the stock price of a globally listed banking enterprise.

Alongside the primary goal, the project also pursues several secondary objectives that serve to deepen the research value and practical applicability of the modeling results. These include quantifying the contribution of each core feature dimension to the predictive accuracy of HSBC's stock price to clarify the key driving factors of its price fluctuations; comparing the predictive performance of regression techniques with increasing complexity against the NPB to quantitatively measure the added value of machine learning models in stock price forecasting; extracting domain-specific and actionable insights into the drivers of HSBC's stock price to provide empirical references for understanding the fluctuation rules of cross-listed global bank stocks; and establishing a standardized, reproducible end-to-end pipeline for financial time-series prediction that covers data collection, cleaning, feature engineering, modeling and evaluation with strict adherence to time-series analysis best practices.

Success for the project is defined by both quantitative and qualitative metrics that collectively reflect the accuracy, validity and practical value of the research.

## **2.1 Quantitative Success Metrics**

The trained models must outperform the NPB on the out-of-sample test set with clear quantitative thresholds: the RMSE of the optimal model is reduced by at least 15% compared with the NPB; the MAE is controlled within 5% of the fluctuation range of HSBC's weekly closing price from 2019 to 2025; the  $R^2$  of the optimal model is no less than 0.3 (adapted to the weak correlation characteristics of financial time-series data); all machine learning models achieve better performance than the OLS baseline in at least two of the three core metrics (RMSE, MAE,  $R^2$ ).

## **2.2 Qualitative Success Metrics**

The project must deliver a clear and quantifiable ranking of feature importance for HSBC's stock price prediction with economically interpretable conclusions about key driving factors; ensure full reproducibility of all project steps from data collection to model deployment with complete code documentation and standardized data output files; strictly validate the absence of lookahead bias, data leakage or overfitting in all data handling and modeling processes with model results in line with the statistical assumptions of time-series analysis; and complete all project deliverables on schedule including cleaned datasets, trained models, evaluation reports and a well-organized version control repository.

### 3. Data Specifications & Classification

All data used in this project is publicly available structured numerical time-series data, with no unstructured text or image data included. The data frequency is set to weekly, aligned with global stock market trading weeks from Monday to Friday, to avoid noise interference from intraday price fluctuations. The target variable for prediction is HSBC Holdings' weekly closing stock price, with the primary trading ticker as 0005.HK and the secondary ticker as HSBC.L, sourced from Yahoo Finance as continuous float data. This variable is lagged by 1 week in data processing to eliminate lookahead bias and align with the forecasting logic of using current week's features to predict next week's price.

Feature data is divided into four theoretically motivated and domain-specific categories, all sourced from Yahoo Finance with mixed float and int data types. HSBC firm-specific data includes weekly close/high/low price, trading volume, 5/10/20-week SMAs, and 14-week RSI and ATR indicators, serving to capture firm-specific price trends, momentum, volatility and liquidity dynamics. Banking sector peer data covers the weekly close price and trading volume of JPMorgan, Citigroup, Standard Chartered, and the financial sector ETF XLF, reflecting banking sector-wide trends and systematic industry risk while providing peer valuation benchmarks. Macroeconomic indicators include weekly closing exchange rates of GBP/USD, HKD/USD, CNY/USD and the US 10-Year Treasury Yield, capturing the impact of exchange rate fluctuations on multi-currency revenue and interest rate changes on net interest margin for global banks. Market sentiment indicators consist of weekly closing prices of S&P500, FTSE100, Hang Seng Index, the VIX Fear Index and gold futures, measuring global market risk appetite, cross-market sentiment spillovers and safe-haven asset demand.

#### 3.1 Preliminary Statistical Overview & Data Scale

The data time range spans from January 1, 2019 to December 31, 2025, covering 7 years of weekly market data that is adjusted for global market holidays to avoid incomplete time-series observations. The expected number of observations is approximately 350 rows, calculated based on 52 trading weeks per year net of public holidays across Hong Kong, London and US markets. HSBC's weekly closing stock price during this period has a preliminary mean of approximately 45 HKD, a standard deviation of 8 HKD, and a fluctuation range of 28–65 HKD, showing mild downward volatility with periodic rebounds. In terms of feature quantity, the project will initially collect 30–40 raw features, and after data cleaning, outlier processing and multicollinearity resolution, condense this to 20–25 effective features by retaining only features with  $VIF < 10$  and preliminary feature importance  $> 0.01$ , to balance model training efficiency, stability and predictive power. The data granularity is weekly, aggregated from daily market data by Yahoo Finance, which filters out intraday price noise and is highly aligned with the project's 1-week-ahead forecasting objective. The expected missing value ratio of all features is below 10%, and no feature has a missing value ratio exceeding 30%, which meets the pre-set imputation criteria.

## 3.2 Key Statistical Characteristics & Targeted Handling Strategies

The collected financial time-series data exhibits several key statistical characteristics that pose challenges to regression modeling, and targeted handling strategies are formulated for each characteristic to ensure model validity.

1. **Strict sequentiality:** The data takes date as the core index and the data order cannot be randomized, requiring the adoption of time-series-specific train/test split with no random shuffling and strict avoidance of lookahead bias in all processing steps.
2. **MAR missing values:** Missing values are mainly caused by global market holidays and are classified as MAR, meaning simple mean/median imputation is rejected in favor of conditional imputation methods that preserve time-series trends.
3. **Skewness and outliers:** The data shows skewness and outlier characteristics, with trading volume exhibiting right skewness and VIX having occasional extreme outliers; no data transformation is applied for tree-based models which are robust to skewness, and extreme outliers are capped rather than deleted to retain the original data context.
4. **Severe multicollinearity:** Severe multicollinearity exists with high correlation among peer bank prices and major global indices, so the project diagnoses this issue via VIF and correlation heatmaps, and drops or combines redundant features to avoid model coefficient instability.
5. **Non-stationarity:** Raw price data is non-stationary with time-varying mean and variance, so percentage returns, as stationary data, are used for linear regression models while raw prices are retained for tree-based models that are robust to non-stationarity.

## 4. Data Handling Protocols

A rigorous, four-stage end-to-end data handling protocol is established for this project, guided by the core principles of accuracy, reproducibility and no data leakage, with each stage featuring clear operational objectives, standardized steps and verifiable deliverables to ensure the quality of model-ready data.

### 4.1 Stage 1: Data Collection

Data collection adheres to standardization and error-handling principles to avoid duplicate, incorrect or incomplete data pulls. The process begins with the definition of a unified master ticker list for all target and feature variables to eliminate data ambiguity, followed by the use of the Python yfinance library for data acquisition with fixed, reproducible parameters including ticker list, start date, end date and data interval. A dedicated data pull function with error handling for API outages or ticker invalidity is written to extract only core columns including Close, High, Low and

Volume, and the multi-index of ticker plus metric is flattened to improve data usability before the cleaned raw data is saved as a CSV file with the date index preserved. Post-collection validation is conducted with three key checks to ensure data quality: verifying the completeness of the 2019–2025 date range with no obvious gaps, confirming the correctness of tickers to avoid misalignment between tickers and target assets, and checking the consistency of core columns to ensure all tickers have Close/High/Low/Volume columns.

## **4.2 Stage 2: Data Cleaning**

Data cleaning adopts a 4-layer systematic protocol aligned with data science and financial time-series best practices, with no in-place data modifications to prevent data loss or errors. Structural cleaning forms the first layer, fixing format and data type errors to standardize the data structure by converting the date index to datetime64 type and setting it as the primary index, casting numerical columns to domain-appropriate types with trading volume as nullable Int64 and price/indicator data as float, and removing redundant columns such as duplicate HSBC data from HSBC.L to retain only the primary ticker 0005.HK data for the target variable. The second layer is missing value handling, following the core principle of preserving time-series trends with a tiered imputation strategy for MAR missing values based on the missing value percentage per column: forward fill is used for columns with less than 5% missing values to retain recent price/indicator trends adapted to holiday-induced missing values, linear interpolation is applied for columns with 5–30% missing values for smoother filling of longer missing gaps, and features with more than 30% missing values are dropped directly as imputation results would be unreliable and may introduce bias. A final check is performed on the target variable, with rows with missing values for 0005.HK\_Close dropped as a last resort to ensure prediction target integrity. Validity cleaning is the third layer, eliminating invalid values and mitigating outlier distortion without destroying time-series context: negative prices and volumes, which are economically impossible, are flagged and replaced with NaN then re-imputed via forward fill, and extreme outliers are capped at the 0.5th/99.5th percentiles for all price/volume/indicator columns with no deletion to preserve the original data context while avoiding model training distortion. The fourth layer is stationarity adjustment, solving the non-stationarity problem of raw stock prices that violates classical linear regression assumptions by calculating the weekly percentage returns of HSBC's closing stock price and adding this as a new stationary feature to the dataset, with the first row with NaN generated by percentage change calculations dropped to ensure data validity for subsequent modeling.

## **4.3 Stage 3: Feature Engineering**

Following the systematic data cleaning, feature engineering is the core step to transform clean raw data into model-ready features, which is tailored to the characteristics of financial time-series and HSBC's business attributes as a global cross-listed bank. This stage encompasses four key tasks to enhance feature predictive power, eliminate multicollinearity and align with forecasting logic.

1. **Technical indicator creation:** Domain-specific technical indicators are created using the Python pandas-ta library to calculate standard indicators for HSBC's firm-specific data that capture price dynamics, including 5/10/20-week SMAs for trend analysis, 14-week RSI for momentum measurement, and 14-week ATR for volatility calculation; rows with NaN generated by indicator calculation due to short calculation windows are dropped to ensure data consistency.

2. **Cross-asset ratio feature construction:** Combining HSBC's characteristics as a global cross-listed bank, four core cross-asset ratio features are built to capture interdependencies between different data types and domain-specific economic logic: the HSBC/XLF ratio reflecting HSBC's relative performance against the global financial sector, the GBP/CNY ratio capturing HSBC's exposure to UK and China market exchange rate risks, the SP500/VIX ratio measuring the balance between global market risk appetite and fear sentiment, and the Yield-Gold ratio reflecting the relative attractiveness of risk-free assets and safe-haven assets.

3. **Multicollinearity resolution:** Multicollinearity is resolved to stabilize model coefficients and improve interpretability by first calculating the correlation matrix of all numerical features and plotting a correlation heatmap to identify high-correlation feature pairs with an absolute correlation coefficient greater than 0.8, then using the VIF from the statsmodels library to diagnose severe multicollinearity with VIF greater than 10 and generating a VIF analysis report saved as CSV, and finally dropping features with VIF greater than 10 to obtain clean, low-multicollinearity feature data.

4. **Train/test split and feature scaling:** Time-series train/test split and feature scaling are completed adhering to the core principle of no lookahead bias, the critical step for valid out-of-sample evaluation: an 80/20 chronological split is adopted with the training set covering 2019–2023 and no random shuffling, the target variable is shifted back by 1 week and feature data is truncated accordingly to ensure one-to-one correspondence between input features and prediction target, StandardScaler from the scikit-learn library is used for feature normalization with the scaler fitted only on the training set and applied to the test set to avoid data leakage, and the scaled training/test feature data and target data are saved as separate CSV files for subsequent model training.

## 4.4 Stage 4: Final Data Validation

Before official model training, a comprehensive five-point final validation is conducted on the model-ready data, and only data that passes all checks is deemed usable for modeling to ensure the validity of subsequent model results. The validation confirms the absence of lookahead bias with correct target variable lagging, scaler fitted only on the training set, and no future data involved in feature engineering; verifies temporal independence with no overlapping dates between the training and test sets to ensure the test set is a true out-of-sample dataset; checks data completeness with all features numerical and no remaining missing values or unprocessed outliers; confirms low multicollinearity with all remaining features having VIF less than 10 meaning severe multicollinearity is effectively resolved; and ensures stationarity compliance with percentage returns used for linear regression models and raw prices

retained for tree-based models that are robust to non-stationarity.

## 5. Project Scope

### 5.1 Core Tasks

The project's core tasks are closely aligned with the end-to-end data handling pipeline and divided into six consecutive phases, each with clear core objectives and verifiable key deliverables to ensure the orderly progress of the project. All deliverables are saved in a standardized format and uploaded to the version control repository in a timely manner.

| Phase                  | Core Tasks                                                                                                                               | Key Deliverables                                                                                                                                         |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1: Data Collection     | 1. Standardized weekly data pull;<br>2. Ticker/date range validation;<br>3. Raw data storage                                             | 1. Raw data CSV file;<br>2. Reproducible data collection code (Python);<br>3. Post-collection validation report                                          |
| 2: Data Cleaning       | 1. 4-layer systematic cleaning;<br>2. Missing value imputation;<br>3. Outlier capping;<br>stationarity adjustment                        | 1. Cleaned data CSV file;<br>2. 4-layer data quality check report;<br>3. Correlation heatmaps;<br>4. VIF analysis report                                 |
| 3: Feature Engineering | 1. Technical/cross-asset feature creation;<br>2. Multicollinearity resolution;<br>3. Time-series train/test split;<br>4. Feature scaling | 1. Model-ready feature set;<br>2. Scaled training/test data (CSV);<br>3. Detailed feature engineering log;<br>4. Feature importance preliminary analysis |
| 4: Model Building      | 1. NPB construction; 2. Training of 5 machine learning regression models;<br>3. Hyperparameter tuning via TSCV;                          | 1. Trained models (pickle files);<br>2. Hyperparameter tuning log;<br>3. Training set performance metrics (RMSE/MAE/R <sup>2</sup> );                    |

|                                        |                                                                                                                                                    |                                                                                                                                                                                                  |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                        | 4. Model saving                                                                                                                                    | 4. Model training code                                                                                                                                                                           |
| 5: Model Evaluation                    | 1. Out-of-sample test set performance analysis;<br>2. Residual diagnosis;<br>3. Feature importance calculation;<br>4. Model comparison             | 1. Test set performance metrics table;<br>2. Residual diagnosis report;<br>3. Feature importance ranking (visualization);<br>4. Model comparison chart                                           |
| 6: Result Synthesis & Model Deployment | 1. Final report writing;<br>2. Presentation slide design;<br>3. Repository organization;<br>4. Project summary;<br>5. Model lightweight deployment | 1. Final project report; presentation slides;<br>2. Well-organized GitHub repository (all code/data/reports);<br>3. Project presentation recording;<br>4. Model prediction API/automated script; |

## 5.2 Hierarchical Model Pipeline

This project leverages a hierarchical model pipeline with escalating complexity, which involves training six models in total—one NPB model and five machine learning regression models. Each model is designed with a well-defined application objective, standardized implementation protocols and hyperparameter tuning strategies, and all models are implemented using mainstream Python machine learning libraries. TSCV is employed for hyperparameter optimization to guard against overfitting and guarantee the model's generalization capability.

The NPB is a non-machine learning forecasting method with a fixed logic: it sets the predicted closing price for the next week equal to the current week's closing price. This model acts as the fundamental benchmark for all machine learning models, enabling the quantification of the marginal added value of predictive modeling. OLS Linear Regression, a fundamental linear model implemented via scikit-learn, requires no hyperparameter tuning. It is used to evaluate the linear correlation between the selected features and HSBC Holdings' weekly closing stock price, and serves as the machine learning baseline for subsequent advanced models.

Ridge Regression is a regularized linear model with an L2 penalty term, where the alpha hyperparameter is tuned through TSCV over the range of 0.001 to 100. This model addresses the overfitting issue of standard linear regression and stabilizes model coefficients in the presence of mild multicollinearity among features. Lasso Regression, another regularized linear model with an L1 penalty term, also undergoes alpha hyperparameter tuning via TSCV. It achieves automatic feature

selection by shrinking the coefficients of irrelevant features to zero, thus eliminating mild to severe multicollinearity in the feature set.

Random Forest Regression is a non-linear ensemble model built on decision tree base learners, with its `n_estimators` and `max_depth` hyperparameters optimized through TSCV. This model excels at capturing non-linear relationships and feature interaction effects in the data, while also demonstrating robustness to data skewness and outliers. XGBoost Regression is a state-of-the-art gradient boosting model, for which the `learning_rate`, `max_depth` and `n_estimators` hyperparameters are tuned via TSCV. It delivers high predictive accuracy for financial time-series data, effectively captures complex non-linear patterns, and boasts strong generalization performance in out-of-sample scenarios.

## 6. Six-Week Project Timeline

The project is scheduled for a total of six weeks, with a clear focus area, key milestones, milestone acceptance criteria and deliverable deadlines for each week. The timeline is designed to balance the depth of each task and on-schedule completion of the project, with weekly progress tracked and recorded via version control to ensure accountability.

### 6.1 Week 1: Data Collection & Structural Cleaning

The first week of the project centers on data collection and structural cleaning.

**Key Milestones:** Raw data successfully pulled and saved; structural errors fixed (date/index/type); missing value percentage assessed; collection code submitted to GitHub (version control).

**Milestone Acceptance Criteria:** Raw data CSV file covers the full 2019-2025 time range with no obvious date gaps; all structural errors are corrected and data types are standardized; missing value ratio of all features is statistically assessed and documented; collection code is runnable and submitted to GitHub with clear comments.

### 6.2 Week 2: Data Validation & Exploratory Data Analysis (EDA)

Week 2 focuses on data validation and exploratory data analysis (EDA).

**Key Milestones:** Outlier capping and stationarity adjustment completed; correlation/VIF analysis finished; EDA visualizations (time-series plots, histograms, boxplots) generated; cleaned data saved to GitHub.

**Milestone Acceptance Criteria:** Outliers are capped at 0.5th/99.5th percentiles and stationarity adjustment is completed for linear models; correlation heatmap and VIF analysis report are generated with clear multicollinearity diagnosis; EDA visualizations are complete and interpretable; cleaned data CSV file is saved to GitHub with a detailed data quality report.

## 6.3 Week 3: Feature Engineering & Train/Test Split

The third week is dedicated to feature engineering and the time-series train/test split of the dataset.

**Key Milestones:** Technical/cross-asset features added to dataset; multicollinearity effectively resolved ( $VIF < 10$ ); time-series train/test split and feature scaling completed; scaled data saved.

**Milestone Acceptance Criteria:** All technical and cross-asset ratio features are added and valid; all remaining features have  $VIF < 10$  with multicollinearity effectively resolved; train/test split is chronological (80/20) with no random shuffling and feature scaling is completed without data leakage; scaled training/test data is saved as separate CSV files with a detailed feature engineering log.

## 6.4 Week 4: Model Building & Hyperparameter Tuning

Week 4 prioritizes model building and hyperparameter tuning for all predictive models.

**Key Milestones:** All 6 models trained and saved as pickle files; hyperparameter tuning process logged in detail; training set performance metrics calculated; modeling code committed to GitHub.

**Milestone Acceptance Criteria:** All 6 models (NPB + 5 machine learning models) are successfully trained and serialized as pickle files; hyperparameter tuning via TSCV is fully logged with clear tuning ranges and results; training set performance metrics (RMSE/MAE/ $R^2$ ) are calculated and documented for each model; modeling code is runnable, well-documented and committed to GitHub.

## 6.5 Week 5: Model Evaluation & Insight Extraction

The fifth week focuses on in-depth model evaluation and actionable insight extraction from the modeling results.

**Key Milestones:** Out-of-sample test set performance analyzed in depth; residual diagnosis completed (no obvious bias); feature importance ranking generated (visualization); detailed model comparison table created.

**Milestone Acceptance Criteria:** Out-of-sample test set performance is analyzed with core metrics and improvement ratios compared with baselines; residual diagnosis is complete with no obvious prediction bias and results in line with time-series statistical assumptions; feature importance ranking is generated with visualizations and economically interpretable insights; a detailed model comparison table is created with comprehensive metrics and model performance analysis.

## 6.6 Week 6: Final Reporting, Model Deployment & Project Presentation

The sixth and final week of the project is focused on final reporting, model lightweight deployment and formal project presentation.

**Key Milestones:** Final project report completed (all sections); presentation slides designed and polished; GitHub repository organized (code/data/reports classified); formal project presentation delivered; model lightweight deployment completed with iteration document.

**Milestone Acceptance Criteria:** Final project report is complete with all sections and meets academic writing standards; presentation slides are polished and clearly communicate project objectives, methods and key findings; GitHub repository is well-organized with clear classification of all code, data and reports; formal project presentation is delivered successfully; model is deployed as a lightweight API/automated script with a detailed deployment and iteration document.

## 7. Resource and Risk Mitigation

### 7.1 Tooling for Data Handling and Modeling

The project uses a set of mature, professional and industry-standard Python libraries and tools for all data handling and modeling tasks, with the development environment set to Python 3.9/3.10 (Anaconda) to ensure environment compatibility. Each tool has clear application purposes, version requirements and strictly followed best practices to ensure the standardization, efficiency and reproducibility of the project. All tools are open-source and compatible with mainstream operating systems including Windows, macOS and Linux to eliminate environment-related barriers.

| Tool/Library     | Core Application Purpose                                    | Key Best Practices                                                                                                |
|------------------|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| `yfinance`       | Financial data collection (tickers/indices/indicators)      | 1. Fixed ticker/date parameters;<br>2. Error handling code for API outages;<br>3. Only extract core columns       |
| `pandas`/`numpy` | Data cleaning, feature engineering and numerical processing | 1. Preserve date index;<br>2. Use nullable data types for missing values;<br>3. Avoid in-place data modifications |
| `statsmodels`    | VIF calculation and multicollinearity diagnosis             | 1. Exclude target variable from VIF analysis;<br>2. Drop non-numerical columns in advance;                        |

|                      |                                                                                |                                                                                                                                                                                                                              |
|----------------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      |                                                                                | 3. Generate structured VIF reports                                                                                                                                                                                           |
| `scikit-learn`       | Feature scaling, train/test split, basic model building (linear/random forest) | <ol style="list-style-type: none"> <li>1. Fit scaler only on training set;</li> <li>2. Use `iloc` for time-based split;</li> <li>3. Implement TSCV for hyperparameter tuning</li> </ol>                                      |
| `pandas-ta`          | Technical indicator creation (SMA/RSI/ATR)                                     | <ol style="list-style-type: none"> <li>1. Use standard window lengths (industry conventions);</li> <li>2. Drop NaN rows generated by indicator calculation</li> </ol>                                                        |
| `xgboost`            | Advanced gradient boosting model training                                      | <ol style="list-style-type: none"> <li>1. Tune `learning_rate` to avoid overfitting;</li> <li>2. Use early stopping during training;</li> <li>3. TSCV for hyperparameter optimization</li> </ol>                             |
| `matplotlib/seaborn` | Data visualization (EDA, correlation heatmaps, feature importance)             | <ol style="list-style-type: none"> <li>1. Standardized plot styles;</li> <li>2. Clear labels/legends; save plots as high-resolution files (PNG/PDF)</li> </ol>                                                               |
| GitHub               | Full-process version control and deliverable storage                           | <ol style="list-style-type: none"> <li>1. Commit code with clear, descriptive comments;</li> <li>2. Track data versions (raw/clean/scaled);</li> <li>3. Use `.gitignore` to exclude large files (datasets/models)</li> </ol> |

## 7.2 Risk Mitigation

Financial time-series data possesses distinctive statistical properties that introduce a host of risks to the modeling workflow, which could disrupt project progress or render model outputs invalid. To address this, the project first identifies each potential risk, clarifies its impact level and occurrence probability, and then designs targeted, actionable mitigation strategies to eliminate or mitigate its effects. All risk mitigation measures are fully integrated into the end-to-end data processing and modeling pipeline, ensuring a proactive approach to risk management throughout the project.

The risk assessment is based on a two-dimensional matrix of **Impact Level (Critical/High/Medium)** and **Occurrence Probability (High/Medium/Low)**.

| Risk Type                | Impact Level | Occurrence Probability | Mitigation Strategies                                                                                                                                                                                                                                               |
|--------------------------|--------------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lookahead bias           | Critical     | Low                    | <ol style="list-style-type: none"> <li>1. Lag target variable before dataset splitting;</li> <li>2. fit scaler only on training set; exclude future data from feature engineering;</li> <li>3. verify absence of bias via residual time-series plots</li> </ol>     |
| Severe multicollinearity | High         | Medium                 | <ol style="list-style-type: none"> <li>1. Diagnose via VIF and correlation heatmaps;</li> <li>2. drop high-VIF features;</li> <li>3. apply Ridge/Lasso regularization to linear models;</li> <li>4. avoid feature redundancy in cross-asset construction</li> </ol> |
| Non-stationarity         | Medium       | High                   | <ol style="list-style-type: none"> <li>1. Use percentage returns (stationary) for linear models;</li> <li>2. retain raw prices for tree-based models (robust to non-stationarity);</li> <li>3. validate stationarity via statistical tests</li> </ol>               |
| API data gaps            | Medium       | Medium                 | <ol style="list-style-type: none"> <li>1. Pre-test API calls for all tickers;</li> <li>2. prepare Alpha Vantage as backup for critical tickers;</li> <li>3. use interpolation for small gaps;</li> <li>4. document all gaps in data quality report</li> </ol>       |
| Overfitting to           | High         | Medium                 | <ol style="list-style-type: none"> <li>1. Use TSCV for</li> </ol>                                                                                                                                                                                                   |

|                       |        |        |                                                                                                                                                                                                                                  |
|-----------------------|--------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| training set          |        |        | hyperparameter tuning;<br>2. limit effective features to 20-25;<br>3. cap model complexity (e.g., max_depth for tree models);<br>4. monitor train/test performance gap in real time                                              |
| Outlier distortion    | Medium | Medium | 1. Cap outliers at 0.5th/99.5th percentiles (no deletion);<br>2. use MAE (robust to outliers) alongside RMSE for evaluation;<br>3. use tree-based models as primary prediction models                                            |
| Project schedule risk | Medium | Medium | 1. Set 1-2 days buffer for each core stage;<br>2. advance parallel development of complementary tasks;<br>3. complete modular code writing to reduce repeated development;<br>4. hold daily 15-minute standups to track progress |

## 7.3 Limitations of Data Handling Tools

The yfinance library, while convenient for pulling financial data from Yahoo Finance, suffers from significant unreliability and instability due to frequent changes in Yahoo's website structure, which can lead to sudden failures, errors, or incomplete data retrieval. To mitigate these issues, the project adopts a best-practice workflow of downloading all required data once at the very beginning of the project (Week 1), saving it immediately to local CSV files, and then working exclusively offline from those saved copies during development and analysis. This approach eliminates repeated API calls, ensures reproducibility, and completely avoids runtime dependency on Yahoo Finance's availability.

## 7.4 Visualization Tools for EDA & Data Validation

In addition to the standard time-series plots, histograms, and boxplots, the

exploratory data analysis (EDA) phase will incorporate a richer set of visualizations to deeply explore data patterns, relationships, and potential regime shifts across the 2019–2025 period, with each visualization type having a clear core analysis objective:

1. **Multi-asset overlay line plots:** Superimpose HSBC's weekly closing stock price with peer bank prices (JPMorgan, Citigroup, Standard Chartered, XLF), major indices (S&P 500, FTSE 100, Hang Seng), and sentiment indicators (VIX, gold futures) on the same chart. It can identify periods of synchronization or divergence between HSBC's price and market benchmarks, especially during major events such as the COVID-19 market crash and interest rate hiking cycles.
2. **Dual-axis plots (price + trading volume):** Combine HSBC closing price (left axis) with trading volume (right axis) in a single plot. It mainly contributed to examine the volume-price linkage and judge whether significant price movements are supported by effective trading volume spikes.
3. **Rolling correlation heatmaps/pairwise scatter matrices:** Focus on the correlation between HSBC returns and key macroeconomic features (e.g., GBP/USD, US 10-Year Treasury Yield), which can be used to reveal the dynamic evolution of correlations between HSBC's price and macro factors over time and identify key time periods with structural correlation changes.